

IDENTIFICAÇÃO DA EQUIPE

Rafael Feltrim

Nº USP: 15942812

Pedro Vitor S Lau

Nº USP: 13837133

Matheus Araújo Alves

Nº USP: 1690609

1. SIGNIFICADO DAS CATEGORIAS DE PADRÕES GOF

O catálogo da *Gang of Four* (Erich Gamma, Richard Helm, Ralph Johnson e John Vlissides) organiza as soluções arquiteturais em três macrocategorias fundamentadas na intenção primordial do problema de design:

Padrões Criacionais

Abstraem o processo de instanciação. Tornam o sistema independente de como seus objetos são criados, compostos e representados, encapsulando o conhecimento sobre classes concretas.

Padrões Estruturais

Focam na composição de classes e objetos para formar estruturas maiores e mais flexíveis. Permitem que interfaces incompatíveis colaborem e criam abstrações limpas sobre subsistemas.

Padrões Comportamentais

Concentram-se nos algoritmos, na atribuição de responsabilidades e na dinâmica de comunicação e controle de fluxo entre objetos em tempo de execução.

2. CATEGORIA A — PADRÕES CRIACIONAIS

1. Singleton

CRIACIONAL

Garante que uma classe possua apenas uma única instância em memória durante todo o ciclo de vida da aplicação e fornece um ponto de acesso global a ela através de um método estático.

Aplicações & Benefícios:

- Gerenciadores de conexão a banco de dados (Connection Pool), Loggers globais e Caches em memória.
- Controle estrito de acesso a recursos compartilhados e inicialização preguiçosa (Lazy Initialization).

Problemas & Riscos:

- Violação do Princípio de Responsabilidade Única (SRP).
- Dificulta testes unitários (acoplamento global impossibilita mocks limpos) e introduz gargalos de concorrência em sistemas multithread.

2. Builder

CRIACIONAL

Separa o processo de construção de um objeto complexo da sua representação final, permitindo que o mesmo algoritmo de construção produza diferentes configurações e representações.

Aplicações & Benefícios:

- Construção de requisições HTTP complexas, geradores de relatórios (PDF/HTML) e entidades com múltiplos parâmetros opcionais.
- Elimina construtores telescópicos e assegura a criação de objetos imutáveis consistentes passo a passo.

Problemas & Riscos:

- Aumenta o volume de código e a quantidade de classes auxiliares no projeto (classes Builder específicas para cada produto).

3. Prototype

CRIACIONAL

Permite a instanciação de novos objetos clonando uma instância existente (protótipo), sem acoplar o código cliente às classes concretas que estão sendo duplicadas.

Aplicações & Benefícios:

- Spawning de entidades em engines de jogos, clonagem de estados complexos em memória e operações de snapshot.
- Evita o custo de inicializações pesadas (como I/O e consultas a banco) ao duplicar instâncias já carregadas.

Problemas & Riscos:

- A clonagem de grafos de objetos complexos contendo referências circulares ou ponteiros profundos (Deep Copy) é complexa e sujeita a bugs.

Tabela Comparativa 1: Padrões Criacionais

Dimensão	Singleton	Builder	Prototype
Mecanismo de Criação	Instanciação única sob demanda encapsulada internamente	Montagem passo a passo via interface fluente de construção	Duplicação em memória a partir de um protótipo existente (Clone)
Foco Central	Garantia de instância única e acesso global coordenado	Controle da complexidade e etapas de montagem do objeto	Evitar o custo de instanciação direta através de cópia
Similitudes	Todos abstraem o uso direto do operador new pelo cliente, desacoplando o consumidor da instanciação de classes concretas.		
Diferenças Chave	Retorna sempre o <i>mesmo</i> objeto existente.	Gera <i>novas instâncias</i> com configurações distintas.	Gera <i>novas instâncias idênticas</i> via cópia profunda.

3. CATEGORIA B — PADRÕES ESTRUTURAIS

4. Adapter

ESTRUTURAL

Converte a interface de uma classe existente em outra interface esperada pelos clientes, permitindo que classes com interfaces incompatíveis colaborem entre si.

Aplicações & Benefícios:

- Integração de bibliotecas legadas ou de terceiros (SDKs de pagamento, exportadores de XML/JSON).
- Reuso de componentes sem alterar o código original, preservando o Princípio Aberto/Fechado (OCP).

Problemas & Riscos:

- Adiciona uma camada intermediária de indireção e pode mascarar incompatibilidades conceituais entre sistemas.

5. Proxy

ESTRUTURAL

Fornece um objeto substituto ou marcador de posição que controla e intercepta o acesso a um objeto de serviço real, implementando a mesmíssima interface.

Aplicações & Benefícios:

- Lazy Loading de recursos pesados (Virtual Proxy), controle de permissões de acesso (Protection Proxy) e Caching de requisições remotas.
- Gerencia o ciclo de vida do objeto de forma totalmente transparente para o cliente.

Problemas & Riscos:

- Pode introduzir latência adicional em chamadas simples e aumenta o número de classes intermediárias no sistema.

6. Facade

ESTRUTURAL

Oferece uma interface simplificada e unificada de alto nível para um subsistema complexo composto por múltiplas classes interdependentes.

Aplicações & Benefícios:

- Inicialização de subsistemas de áudio/gráficos, orquestração de APIs de processamento de compras e SDKs unificados.
- Reduz drasticamente as dependências em malha, isolando os clientes da complexidade interna do subsistema.

Problemas & Riscos:

- Risco de a classe Facade tornar-se excessivamente acoplada a todas as engrenagens do subsistema caso não haja modularização adequada.

Tabela Comparativa 2: Padrões Estruturais

Dimensão	Adapter	Proxy	Facade
Interface Exposta	Converte uma interface existente em <i>outra interface diferente</i>	Mantém <i>exatamente a mesma interface</i> do objeto real	Define uma <i>nova interface simplificada</i> de alto nível
Objetivo Primário	Compatibilizar interfaces que não conversavam entre si	Controlar, atrasar, proteger ou auditar o acesso ao objeto	Simplificar o uso de uma malha densa de classes e subsistemas
Similitudes	Todos atuam como intermediários através de composição estrutural, encapsulando objetos subjacentes para proteger o cliente de complexidades.		
Diferenças Chave	Muda o formato da chamada para adequação de contrato.	Não altera contrato; apenas intercepta a execução.	Agrupar múltiplos subsistemas em um único ponto de entrada.

4. CATEGORIA C — PADRÕES COMPORTAMENTAIS

7. Mediator

COMPORTAMENTAL

Centraliza as interações complexas entre múltiplos objetos em um único objeto mediador, eliminando as dependências diretas entre os colegas.

Aplicações & Benefícios:

- Diálogos de interface gráfica com componentes interdependentes, torres de controle e orquestração de microserviços.
- Reduz o acoplamento $N \times M$ e permite reutilizar objetos colegas isoladamente.

Problemas & Riscos:

- O Mediador corre o risco de virar um *God Object* monolítico com regras de negócio em excesso.

8. Observer

COMPORTAMENTAL

Define um mecanismo de assinatura 1:N no qual a alteração de estado em um objeto publicador notifica e atualiza automaticamente todos os assinantes registrados.

Aplicações & Benefícios:

- Feeds de redes sociais, notificações "avise-me quando chegar", sistemas reativos e event listeners.
- Elimina a necessidade de *Polling* ativo e respeita o Princípio Aberto/Fechado (OCP).

Problemas & Riscos:

- Ordem de notificação não determinística e risco de vazamentos de memória (Lapsed Listener Problem).

9. Chain of Responsibility

COMPORTAMENTAL

Encadeia uma série de manipuladores sequenciais, passando a requisição ao longo da cadeia até que um deles processe o pedido ou a esteira seja finalizada.

Aplicações & Benefícios:

- Middlewares HTTP (autenticação, autorização, validação de payload) e rotinas de aprovação em camadas.
- Desacopla o emissor do receptor e permite reordenar ou compor novos filtros em tempo de execução.

Problemas & Riscos:

- Não há garantia de que a requisição seja processada caso nenhum nó na esteira a trate.

Tabela Comparativa 3: Padrões Comportamentais

Dimensão	Mediator	Observer	Chain of Responsibility
Topologia do Fluxo	Estrela ($N \rightarrow 1 \rightarrow M$): Comunicação centralizada	Broadcast ($1 \rightarrow N$): Publicador emite para lista de ouvintes	Pipeline Linear ($1 \rightarrow H_1 \dots \rightarrow H_n$): Encadeamento sequencial
Destinatário	O Mediador decide dinamicamente quem acionar	Todos os observadores registrados são acionados	O primeiro nó apto trata ou interrompe o fluxo
Similitudes	Todos buscam desacoplar emissores e receptores através de polimorfismo, flexibilizando a atribuição de responsabilidades em tempo de execução.		
Diferenças Chave	Comunicação bidirecional coordenada por um árbitro.	Difusão unidirecional reativa sem coordenação de retorno.	Encaminhamento sequencial passo a passo até resolução.